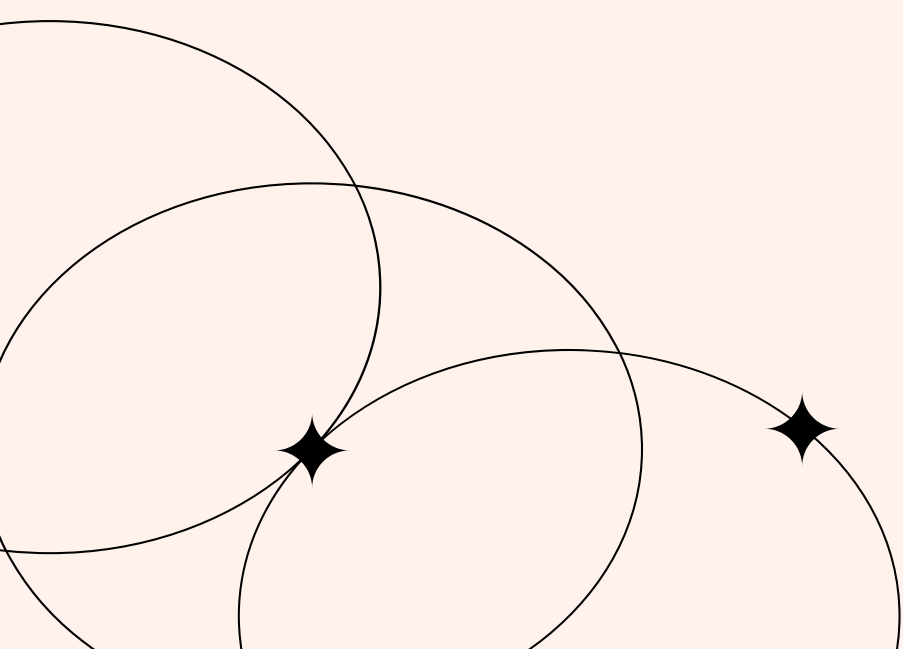
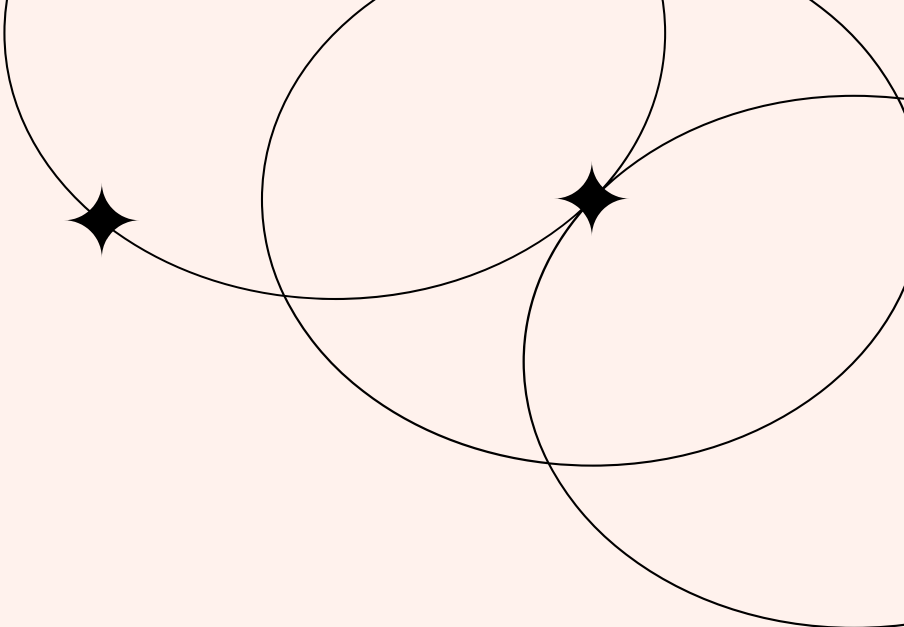




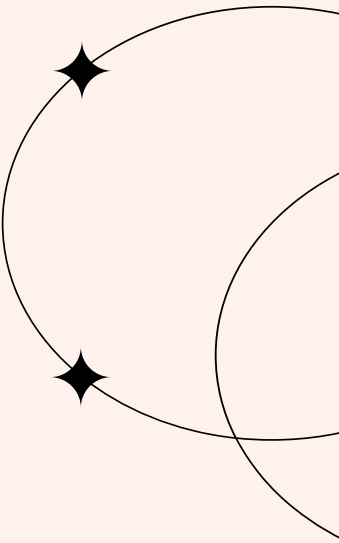
Turso – distribuirana baza podataka za cloud i edge

Luka Veličković, 2101

SQLite



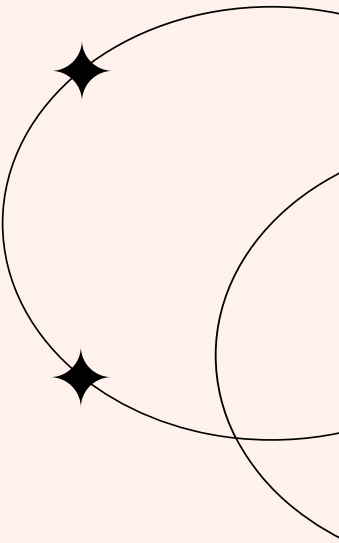
- Self-contained
 - veoma mali broj zavisnosti (par sistemski poziva)
 - bez eksternih zavisnosti
 - jedan source fajl
- Serverless
 - serverless – ne izvršava se na posebnoj mašini, već u istom threadu ili procesu kao i sama aplikacija
- Zero-configuration – ne postoji proces instalacije ili podešavanja, kreiranje nove instance ili dodeljivanje permisija
- Transactional – nudi ACID svojstva i implementira serializable nivo izolacije
 - U jednom trenutku samo 1 writer može da upisuje u fajl
 - 2 moda – journal mod i WAL mod



libSQL



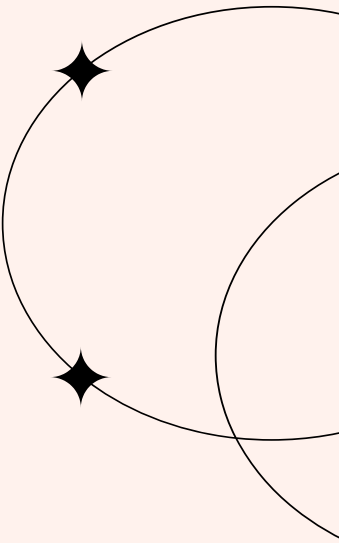
- Fork SQLite projekta, pod okriljem kompanije Turso
- SQLite je open-source, ali ne i open-contribution
- Dodaci
 - ALTER TABLE kod SQLite ograničena na preimenovanje i dodavanje ili uklanjanje postojećih kolona. libSQL omogućava promenu type affinity-ja, dodavanje i uklanjanje ograničenja
 - Definisanje funkcija korišćenjem web assembly-ja direktno u bazi
 - Virtuelni WAL – obezbeđuje API za implementaciju custom WAL-a
 - Native vector similarity search



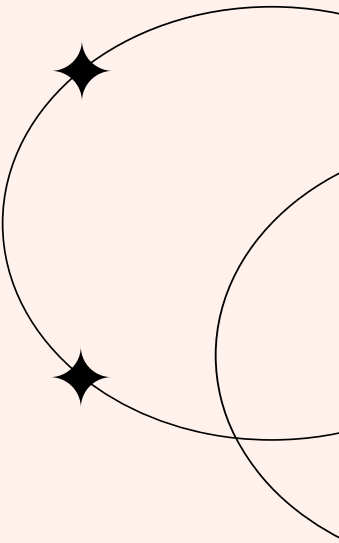
Turso (baza)



- Kompletan rewrite SQLite baze od nule, u Rust-u
 - Modernija arhitektura, asinhroni IO...
- CDC
- Konkurentni upisi
- Sync



Turso CDC



```
turso> CREATE TABLE transactions (  
  id INTEGER PRIMARY KEY,  
  account_id INTEGER NOT NULL,  
  amount INTEGER NOT NULL, -- cents  
  description TEXT,  
  transaction_date TEXT NOT NULL,  
  created_at TEXT NOT NULL,  
  status TEXT NOT NULL DEFAULT 'posted', -- pending, posted, failed, reversed  
  
  FOREIGN KEY (account_id) REFERENCES accounts(id)  
);
```


Turso CDC

```
import turso
from datetime import datetime, timezone

con = turso.connect("/home/lule/turso/luka.db")
cur = con.cursor()

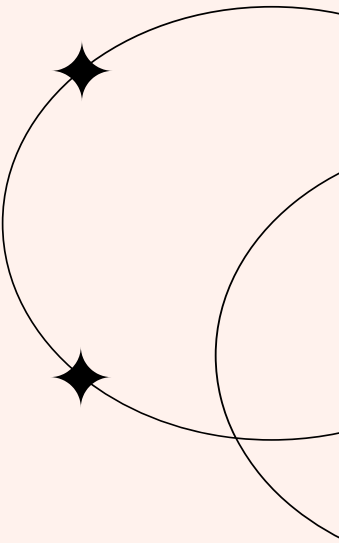
cur.execute("PRAGMA capture_data_changes_conn('full');")
cur.execute("PRAGMA foreign_keys = ON;")

now = datetime.now(timezone.utc).replace(microsecond=0).isoformat().replace("+00:00", "Z")
tx_id = 1001

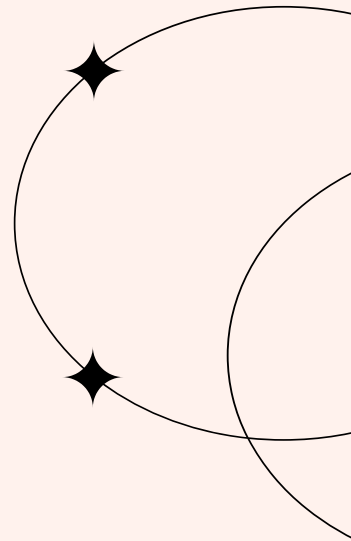
cur.execute(
    """
    INSERT INTO transactions (id, account_id, amount, description, transaction_date, created_at, status)
    VALUES (?, ?, ?, ?, ?, ?, ?)
    """,
    (tx_id, 1, -2599, "Coffee and lunch", "2026-06-29", now, "pending",),
)

cur.execute(
    """
    UPDATE transactions
    SET status = ?
    WHERE id = ?
    """,
    ("posted", tx_id),
)

con.commit()
```



Turso CDC

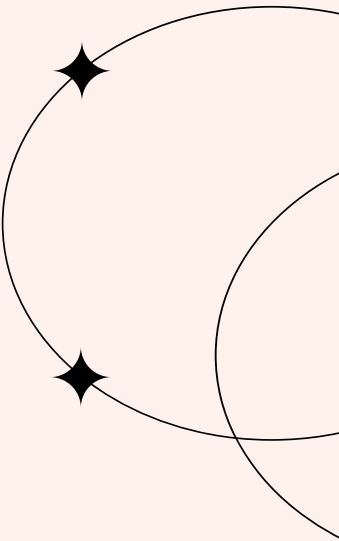


```
turso> select * from turso_cdc;
```

change_id	change_time	change_txn_id	change_type	table_name	id	before	after	updates
1	17827...	1	1	trans...	1001		-...	
2	17827...	1	0	trans...	1001	-...	-...	
3	17827...	1	2					

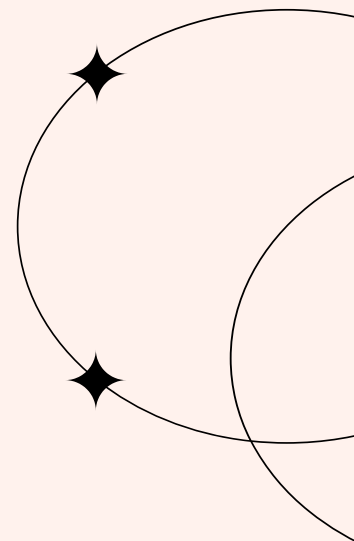
Turso CDC

```
SELECT
  change_id,
  change_txn_id,
  CASE change_type
    WHEN 1 THEN 'INSERT'
    WHEN 0 THEN 'UPDATE'
    WHEN -1 THEN 'DELETE'
    WHEN 2 THEN 'COMMIT'
  END AS operation,
  table_name,
  id,
  bin_record_json_object(
    table_columns_json_array('transactions'),
    before
  ) AS before_state,
  bin_record_json_object(
    table_columns_json_array('transactions'),
    after
  ) AS after_state
FROM turso_cdc
WHERE table_name = 'transactions'
ORDER BY change_id;
```



change_id	change_txn_id	operation	table_name	id	before_state	after_state
1	1	INSERT	transactions	1001		{"id":1001,"account_id":1,"amount":-2599,"description":"Cof...
2	1	UPDATE	transactions	1001	{"id":1001,"account_id":1,"amount":-2599,"description":"Coff...	{"id":1001,"account_id":1,"amount":-2599,"description":"Cof...

Turso CDC



```
CREATE TABLE products (id INTEGER PRIMARY KEY, name TEXT);
ALTER TABLE products ADD COLUMN price INTEGER NOT NULL DEFAULT 0;
DROP TABLE products;
```

change_id	change_txn_id	operation	table_name	id	before_state	after_state
4	4	INSERT	sqlite_schema	7		{"type": "table", "name": "products", "tbl_name": "products", "rootpage": 8, "sql": "CREATE TABLE products (id INTEGER PRIMARY KEY, name TEXT)"}
6	6	UPDATE	sqlite_schema	7	{"type": "table", "name": "products", "tbl_name": "products", "rootpage": 8, "sql": "CREATE TABLE products (id INTEGER PRIMARY KEY, name TEXT)"}	{"type": "table", "name": "products", "tbl_name": "products", "rootpage": 8, "sql": "CREATE TABLE products (id INTEGER PRIMARY KEY, name TEXT, price INTEGER NOT NULL DEFAULT 0)"}
8	8	DELETE	sqlite_schema	7	{"type": "table", "name": "products", "tbl_name": "products", "rootpage": 8, "sql": "CREATE TABLE products (id INTEGER PRIMARY KEY, name TEXT)"}	

```
{
  "type": "table",
  "name": "products",
  "tbl_name": "products",
  "rootpage": 8,
  "sql": "CREATE TABLE products (id INTEGER PRIMARY KEY, name TEXT)"
}

{
  "type": "table",
  "name": "products",
  "tbl_name": "products",
  "rootpage": 8,
  "sql": "CREATE TABLE products (id INTEGER PRIMARY KEY, name TEXT, price INTEGER NOT NULL DEFAULT 0)"
}
```

Konkurentni upisi

```
def run_worker(db_path: str, mode: str, worker_id: int, start_at: float) -> tuple[int, float, str]:
    conn = turso.connect(db_path)

    try:
        if mode == "mvcc":
            conn.execute("PRAGMA journal_mode = 'mvcc'").fetchone()

        t0 = time.perf_counter()

        try:
            if mode == "mvcc":
                conn.execute("BEGIN CONCURRENT")
            else:
                conn.execute("BEGIN IMMEDIATE")

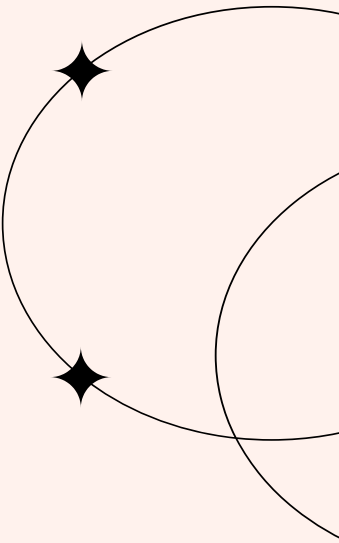
            conn.execute("INSERT INTO hits VALUES (?)", (worker_id,))
            time.sleep(SLEEP_INSIDE_TX)
            conn.execute("COMMIT")

            elapsed = time.perf_counter() - t0
            return worker_id, elapsed, "committed"

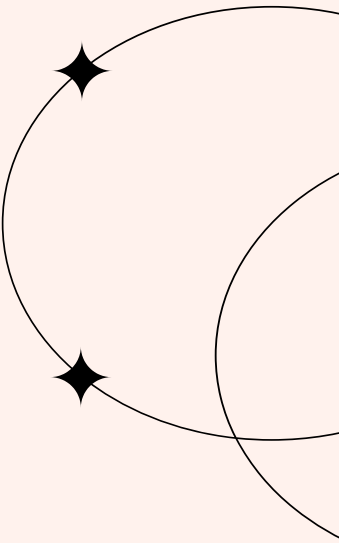
        except Exception as e:
            try:
                conn.execute("ROLLBACK")
            except Exception:
                pass

            elapsed = time.perf_counter() - t0
            return worker_id, elapsed, f"failed: {type(e).__name__}: {e}"

    finally:
        conn.close()
```



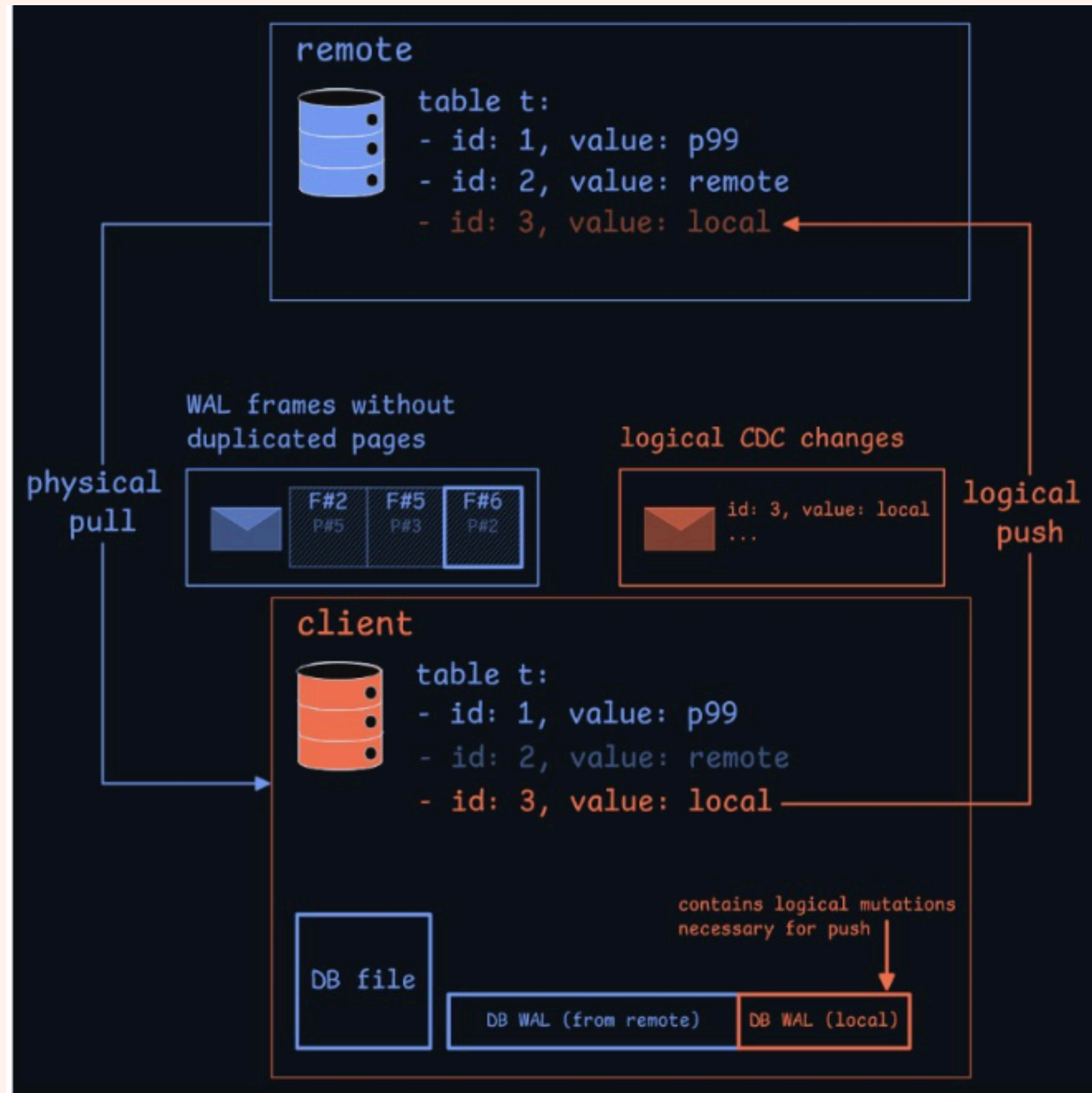
Konkurentni upisi



```
=====
mode: normal
rows inserted: 1
wall time: 0.260s
=====
worker=0 elapsed=0.257s committed
worker=1 elapsed=0.000s failed: OperationalError: database is locked
worker=2 elapsed=0.000s failed: OperationalError: database is locked
worker=3 elapsed=0.000s failed: OperationalError: database is locked
worker=4 elapsed=0.000s failed: OperationalError: database is locked
worker=5 elapsed=0.000s failed: OperationalError: database is locked
worker=6 elapsed=0.000s failed: OperationalError: database is locked
worker=7 elapsed=0.000s failed: OperationalError: database is locked

=====
mode: mvcc
rows inserted: 8
wall time: 0.276s
=====
worker=0 elapsed=0.257s committed
worker=1 elapsed=0.259s committed
worker=2 elapsed=0.255s committed
worker=3 elapsed=0.263s committed
worker=4 elapsed=0.256s committed
worker=5 elapsed=0.256s committed
worker=6 elapsed=0.261s committed
worker=7 elapsed=0.257s committed
```

Turso sync

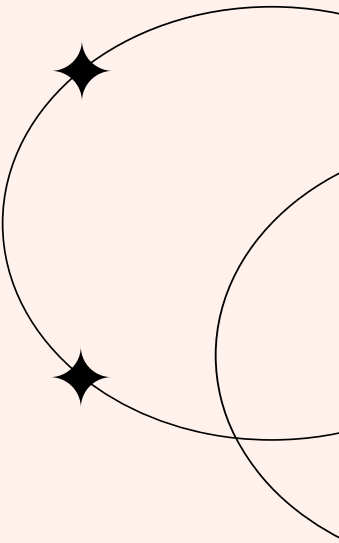


```
tursodb ./server.db --sync-server 0.0.0.0:8080
```

```
import turso.sync
```

```
conn = turso.sync.connect(  
    path="./local-replica.db",  
    remote_url="http://localhost:8080",  
)
```

Turso sync



```
import turso.sync

client_a = turso.sync.connect(
    path="./client-a.db",
    remote_url="http://localhost:8080",
)

client_a.execute("CREATE TABLE IF NOT EXISTS notes (id TEXT PRIMARY KEY, body TEXT)")
client_a.commit()
client_a.execute("INSERT INTO notes VALUES ('n1', 'hello from client A')")
client_a.commit()

client_a.push()
```

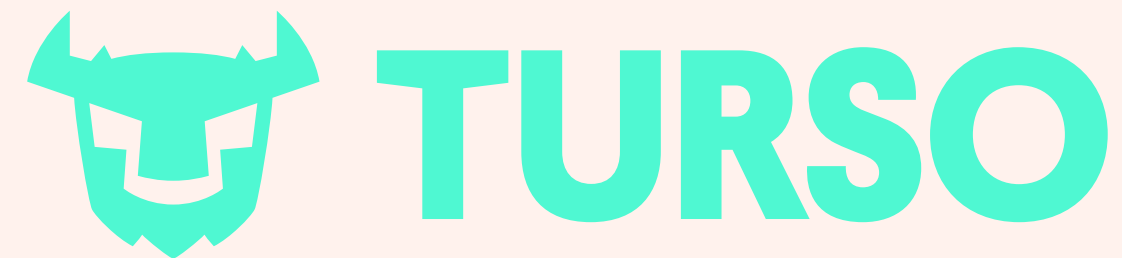
```
import turso.sync

client_b = turso.sync.connect(
    path="./client-b.db",
    remote_url="http://localhost:8080",
)

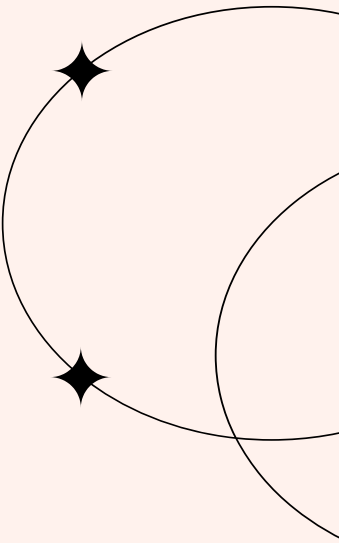
changed = client_b.pull()

rows = client_b.execute("SELECT * FROM notes").fetchall()
print(rows)
# [('n1', 'hello from client A')]
```


Turso cloud

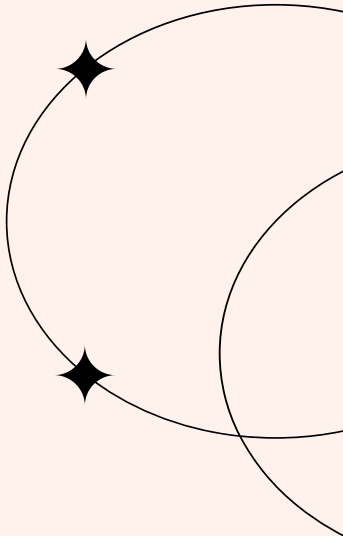


- SQLite (libSQL/Turso) fajlovi se skladište u AWS S3 i AWS S3 express
 - Svaka SQLite-kompatibilna baza ima dve glavne komponente
 - WAL fajl
 - Glavni fajl baze
 - Novi commitovi se skladište u AWS S3 express (performanse bliske fajl sistemu) i periodično se upisuju u AWS S3
 - Garantuje 99.999999999% durability (ista garancija kao AWS S3)

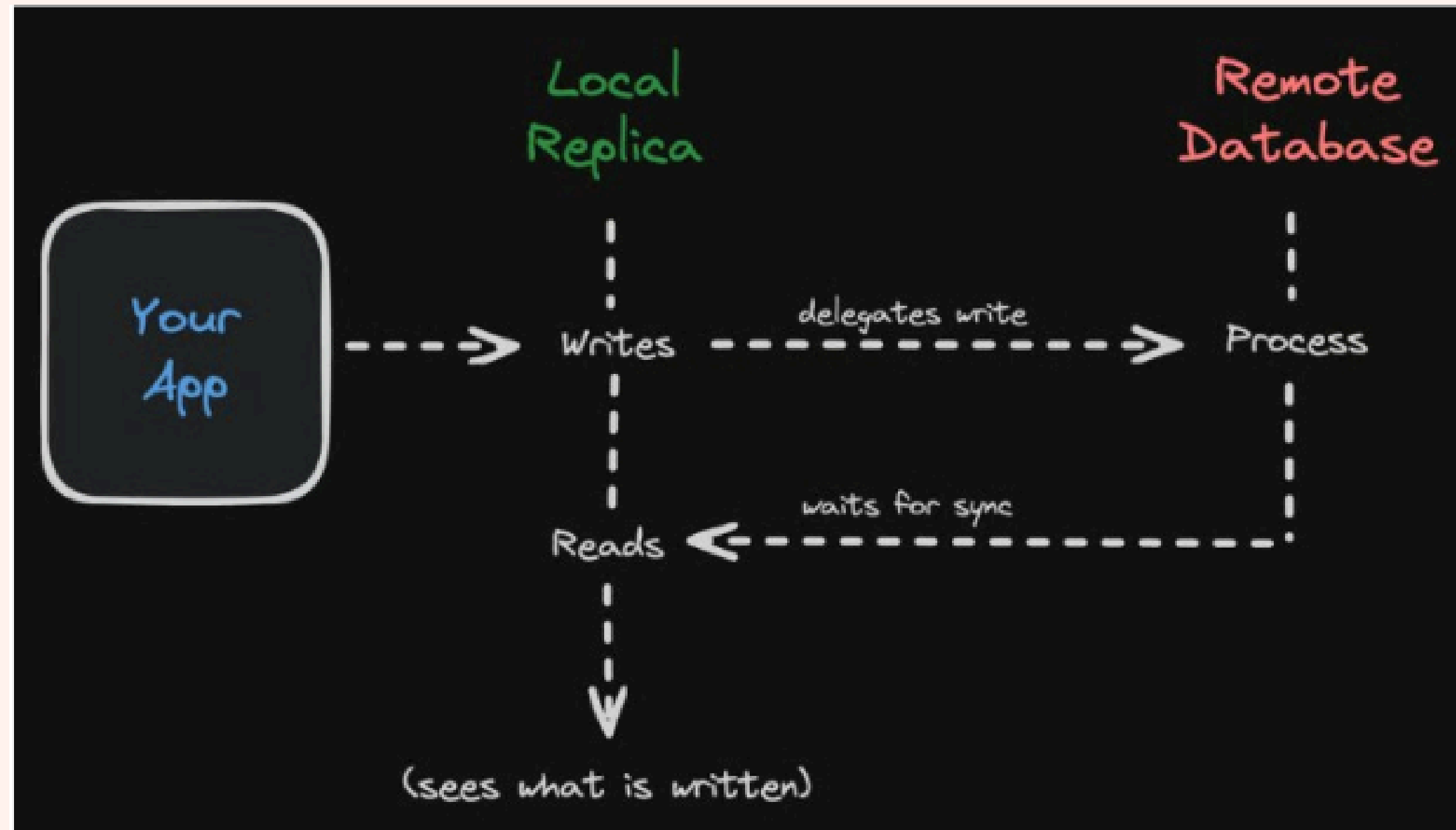


Turso cloud

	Free	Developer	Scaler	Pro
	\$0	\$4.99 /month	\$24.92 /month	\$416.58 /month
	Sign up	Get Developer	Get Scaler	Get Pro
Databases	100	∞ Unlimited	∞ Unlimited	∞ Unlimited
Storage	5GB	9GB + \$0.75 / GB	24GB + \$0.50 / GB	50GB + \$0.45 / GB
Monthly Rows Read	500 Million	2.5 Billion + \$1 / Billion	100 Billion + \$0.80 / Billion	250 Billion + \$0.75 / Billion
Monthly Rows Written	10 Million	25 Million + \$1 / Million	100 Million + \$0.80 / Million	250 Million + \$0.75 / Million
Monthly Syncs	3GB	10GB + \$0.35 / GB	24GB + \$0.25 / GB	100GB + \$0.15 / GB
Point-In-Time Restore	✓ 1 day	✓ 10 days	✓ 30 days	✓ 90 days



Turso cloud – embedded replicas



Turso cloud – embedded replicas

```
▼ insert into notes
  values (1, 'Embedded replica note', 'Embedded replica will be replaced with sync', CURRENT_TIMESTAMP);

▼ insert into notes
  values (2, 'Turso is based on libSQL', 'Turso uses libSQL, which is a fork of SQLite designed for modern cloud and edge use cases.', CURRENT_TIMESTAMP);

▼ insert into notes
  values (3, 'Turso supports embedded replicas', 'Embedded replicas keep a local database file for fast local reads while writes are sent to the cloud');

▼ insert into notes
  values (4, 'Embedded replicas are not fully offline-first', 'With embedded replicas, reads are local, but writes still require a connection to the cloud');

▼ insert into notes
  values (5, 'Turso Sync is local-first', 'Turso Sync allows reads and writes to happen locally, then changes can be pushed to and pulled from Turso Cloud');

▼ insert into notes
  values (6, 'Turso Cloud can coordinate synchronization', 'Turso Cloud can act as the remote coordination point that keeps local database files in sync');

▼ insert into notes
  values (7, 'Turso is useful at the edge', 'Because data can be read from a local replica, Turso is useful for applications where low latency is important');

▼ insert into notes
  values (8, 'Turso differs from Postgres', 'A traditional Postgres app usually depends on a remote database server, while Turso can support local replicas');

▼ insert into notes
  values (9, 'SQLite developer experience', 'Turso keeps much of the simplicity of SQLite while adding cloud features such as replication, authentication, and backups');
```

Turso cloud - embedded replicas

```
import libsql

conn = libsql.connect(
    "local-replica.db",
    sync_url="libsql://maindb-lulexs.aws-eu-west-1.turso.io",
    auth_token="eyJhbGciOiJIJFZERTQSIjInR5cCI6IkpXVCJ9.eyJhIjoicnciLCJleHAiOjE3ODMyNTE3NzMsImIhdCI6MTc4MzE2NTM3MywiaWQiOiIwMTlmM"
)

conn.sync()

conn.execute(
    "INSERT INTO notes VALUES (?, ?, ?, ?)",
    ("10", "Offline demo", "This app reads locally but writes through Turso Cloud.",
    , "CURRENT_TIMESTAMP")
)

conn.commit()

rows = conn.execute("SELECT count(*) FROM notes").fetchall()
print(rows)
```

Turso cloud - embedded replicas

```
import libsql

conn = libsql.connect(
    "local-replica.db",
)

conn.execute(
    "INSERT INTO notes VALUES (?, ?, ?, CURRENT_TIMESTAMP)",
    ("12", "Local write", "This write is local with no primary in cloud")
)

conn.commit()
```

```
import libsql

conn = libsql.connect(
    "local-replica.db",
    sync_url="libsql://maindb-lulexs.aws-eu-west-1.turso.io",
    auth_token="eyJhbGciOiJIJFZERTQSI6InR5cCI6IkpXVCJ9.eyJhbnVudCI6ImNldHdCI6MTc4MzE2NTM3MywiaWQiOiIwMTlmMj"
)

conn.sync()

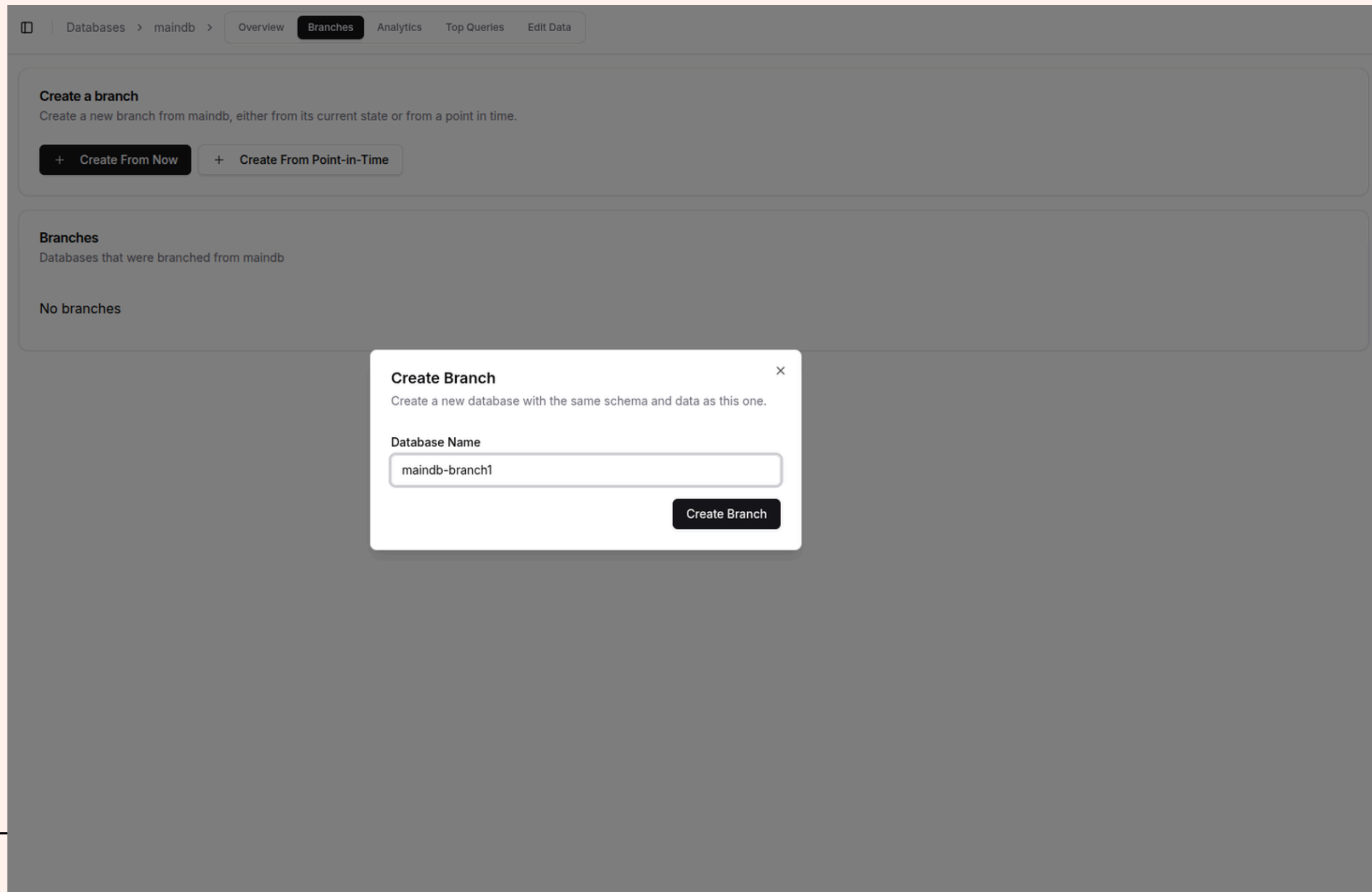
rows = conn.execute("SELECT * FROM notes where id = 12").fetchall()
print(rows)
```

Turso cloud – embedded replicas

```
(venv) lule@lule-Asus-Vivobook:~/Desktop/DBMS/P3/impl$ python embedded_replicas.py
[(12, 'Local write', 'This write is local with no primary in cloud', '2026-07-04 12:05:02')]
(venv) lule@lule-Asus-Vivobook:~/Desktop/DBMS/P3/impl$ █
```

☐	id	title	body	updated_at
☐	1	Embedded replica note	Embedded replica will be rep...	2026-07-04 11:37:29
☐	2	Turso is based on libSQL	Turso uses libSQL, which is ...	2026-07-04 11:39:23
☐	3	Turso supports embedded repl...	Embedded replicas keep a loc...	2026-07-04 11:39:24
☐	4	Embedded replicas are not fu...	With embedded replicas, read...	2026-07-04 11:39:25
☐	5	Turso Sync is local-first	Turso Sync allows reads and ...	2026-07-04 11:39:26
☐	6	Turso Cloud can coordinate s...	Turso Cloud can act as the r...	2026-07-04 11:39:27
☐	7	Turso is useful at the edge	Because data can be read fro...	2026-07-04 11:39:27
☐	8	Turso differs from Postgres	A traditional Postgres app u...	2026-07-04 11:39:28
☐	9	SQLite developer experience	Turso keeps much of the simp...	2026-07-04 11:39:29
☐	10	Offline demo	This app reads locally but w...	CURRENT_TIMESTAMP

Turso cloud – branching



Turso cloud – branching

SQL console

Schema

Search...   

notes

 DATA  STRUCTURE     Filters  Sort  Columns 

141ms

< 1 - 12 of 12 >

☐	id integer	title text	body text	updated_at text	+
☐	1	Embedded replica note	Embedded replica will ...	2026-07-04 11:37:29	
☐	2	Turso is based on libS...	Turso uses libSQL, whi...	2026-07-04 11:39:23	
☐	3	Turso supports embedde...	Embedded replicas keep...	2026-07-04 11:39:24	
☐	4	Embedded replicas are ...	With embedded replicas...	2026-07-04 11:39:25	
☐	5	Turso Sync is local-fi...	Turso Sync allows read...	2026-07-04 11:39:26	
☐	6	Turso Cloud can coordi...	Turso Cloud can act as...	2026-07-04 11:39:27	
☐	7	Turso is useful at the...	Because data can be re...	2026-07-04 11:39:27	
☐	8	Turso differs from Pos...	A traditional Postgres...	2026-07-04 11:39:28	
☐	9	SQLite developer exper...	Turso keeps ...   	2026-07-04 11:39:29	
☐	10	Offline demo	This app reads locally...	CURRENT_TIMESTAMP	
☐	11	Offline demo	This app reads locally...	CURRENT_TIMESTAMP	
☐	12	Branch write	This write happend on ...	2026-07-04 15:35:00	

Powered by  drizzle studio